Cloud Computing Mini Project — Report

Knowledge Distillation & Deployment of a Command-Parsing Language Model on a Local Cloud

Student: Abhishek Yadav **Roll:** 230103041 **Final tag:** `final`

Repo: https://github.com/ABHISHEK27Y/CS3081-CLOUD-PROJECT.git

Note: numbers below are measured in the local k3d cluster and saved under `results/`. Text under *Assumption* is a stated assumption, not a measured result.

Architecture overview

The system has four planes on a two-node k3d (k3s-in-Docker) cluster on one laptop: **storage** (SeaweedFS, S3 API, namespace `data`), **pipeline** (labelling workers, namespace `pipeline`), **serving** (FastAPI `/parse`, namespace `app`, behind a Traefik Ingress) and **control** (Makefile + GitHub Actions). The teacher (`phi3.5` on Ollama) labels commands; `clean.py` produces `datasets/v1/train.jsonl`; `pythia-70m` is fine-tuned on those labels; weights live in object storage and a GitHub Release (never in git or the image); the service downloads them at start-up. Data flow: `commands_pool` → `teacher shards` → `clean` → `train.jsonl` → `student` → `models/` → `service` → `/parse`.

Task 1 — Object storage & parallel teacher labelling

Architecture. `datasets` and `models` buckets on SeaweedFS. An **indexed Job** runs two workers; Kubernetes gives each a `JOB_COMPLETION_INDEX` used as `SHARD_INDEX` (`SHARD_COUNT=2`).

Implementation. `worker/label_worker.py` takes commands where `line_number % SHARD_COUNT == SHARD_INDEX`, calls Ollama (temp 0, seed 42) with `prompts/teacher_prompt.txt` and writes {`id,text,teacher_raw`} to `datasets/teacher/shard-N.jsonl` (raw reply stored unedited, with resume + periodic upload). `worker/clean.py` removes (a) invalid JSON, (b) wrong keys, (c) bad intent, (d) slot value not present in the command, (e) duplicate text (smallest id kept), writing `datasets/v1/train.jsonl` in id order with its SHA-256.

Results.

input	(a) invalid	(b) keys	(c) intent	(d) slot	duplicates	kept
3000	0	3	11	192	46	**2748**

Checksum `datasets/v1/train.sha256` = `153368ded71b3d1f736f4af55f735a257840eb1c4f6bd220aebd1a908d0b9787`. (input = $a + b + c + d$ + dup + kept → $0 + 3 + 11 + 192 + 46 + 2748 = 3000$, verified; checksum is deterministic on re-run.) With JSON-constrained decoding (`format:"json"`) the teacher produced **100%** valid JSON and **0** empty replies over all 3,000 commands, so rejections (a) are zero and cleaning is dominated by (d) invented-slot and duplicate removal.

Sharding. Deterministic modulo split → every id appears in exactly one shard, no coordination needed, and shards run on separate pods in parallel; adding workers only changes `SHARD_COUNT`.

Limitations / improvements. CPU teacher is slow (131 min for 3,000 commands on 2 parallel shards, laptop CPU); a shared work queue would rebalance stragglers better than static modulo sharding.

Task 2 — Training the student (sequence-level distillation)

Architecture. Each example is `Command: <cmd>\nJSON: <teacher JSON>`; the student learns to reproduce the teacher's output (sequence-level). Prompt tokens are masked in the loss so only the JSON is learned.

Implementation. `training/train.py` reads only `training/config.yaml` (seed 42, lr 5e-5, batch 8, 4 epochs, max_seq_len 128, first 1,500 records). `make train SUBSET=50` smoke-tests. Loss per epoch → `training/loss.csv`. Model + tokenizer → `models/v1/`, archived to `v1.tar.gz` with SHA-256; v2 uses a different seed.

Results. (percentages over the 100-example `validation.jsonl`)

model	json_validity	intent	date	time	place	full_frame
teacher (phi3.5)	100.0	86.0	80.0	86.0	87.0	**52.0**
untrained pythia-70m	0.0	0.0	0.0	0.0	0.0	**0.0**
student v1	100.0	84.0	83.0	88.0	85.0	**50.0**
student v2 (seed 7)	96.0	78.0	63.0	79.0	74.0	**33.0**

Loss per epoch (student v1): **0.247** → **0.101** → **0.063** → **0.061** (`results/loss_v1.csv`; v2 in `loss_v2.csv`). After distillation the student reaches **100% valid JSON** and **full_frame 50%** vs the teacher's 52% — it keeps most of the teacher's skill but a 70 M model underfits the slot rules, as expected.

Prompt boundary. Training and inference end the prefix at `"JSON:"`, with the leading space owned by the completion. This makes the prompt tokens a stable prefix of the training sequence; the same boundary is used in `train.py`, `predict.py`, and the service.

Explanation (required). (a) Sequence-level distillation trains on the teacher's *generated text*. (b) Logit-based distillation trains on the teacher's *full probability distribution* per token (soft targets). (c) It is not possible here

because the teacher is served through Ollama, which returns text only — we never see its logits, and the vocabularies differ.
Limitations / improvements. Student full_frame (50%) trails the teacher (52%); one valid JSON reply used an intent outside the eight allowed by the service schema. A larger student or cleaner teacher prompt could help. True logit-level distillation would need a same-vocabulary teacher whose logits we can read.

Task 3 — Inference service in a container

Architecture. FastAPI, port 8000: `POST /parse`, `GET /healthz`, `GET /readyz`. Config from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS`; AWS creds from a Secret. Model downloaded from object storage at start-up (loaded in a background thread → `/readyz` 503 until ready).
Implementation. Every output is validated against `schema.json`; invalid → HTTP 500 {`"error":"invalid_model_output"`}, never broken JSON with 200. `FAULT_INJECT=1` forces an invalid output to test that path. Headers `X-Model-Version`, `X-Served-By` (Pod name). CPU-only pinned image, weights excluded.
Results. The live v1 service downloaded seven model objects from SeaweedFS at startup; `/readyz` returned 503 during loading and 200 afterward. Ingress `/parse` returned the four-key body and both required headers. With `FAULT_INJECT=1`, `/parse` returned the exact HTTP 500 body {`"error":"invalid_model_output"`}. The built image is **1.742 GB** (< 2 GB). In a container limited to 1 CPU and 1 GiB, **50 sequential requests** completed in 36.9 s; 1 returned an invalid-intent 500, observed peak memory was **344.5 MiB**, and Docker reported `OOMKilled=false` (`results/cluster_50_requests.txt`).
Limitations / improvements. Single uvicorn worker per replica; request batching would raise throughput.

Task 4 — Kubernetes deployment, scaling, reliability

Architecture. Deployment (2 replicas, requests 250m/512Mi, limits 1/1Gi), Service, Traefik Ingress on `/parse`, HPA 1–2 @ 70% CPU, `RollingUpdate maxUnavailable=0, maxSurge=1`. The local cluster has two k3d nodes on one laptop.
Results.

- **Rolling update** v1→v2 under the four-user load test, then rollback: `X-Model-Version` showed v2 after rollout and v1 after `rollout undo`. With a 10 s `preStop` drain, 6 of 238 requests failed (2.52%), all HTTP 500 from invalid model output; there were no Ingress 502s. The 0.5% target was *not* met because of model errors.
- **Recovery:** deleted one Pod under four-user load; the Deployment restored **2 Ready** replicas. Of 252 requests, 3 failed (1.19%), all model-output 500s and no Ingress errors; the 2% failure target was met.
- **Cold start** (3 runs): **20 s, 20 s, 21 s** → median **20 s**; logs show each replacement Pod downloaded seven model objects before `/readyz` became 200.
- **Load balancing** (2 Ready): **129 / 123** header-counted responses → 51.2% / 48.8% across replicas.
- **HPA:** observed **1 → 2 → 1** replicas; CPU peaked at **312%** of request during recovery load (target 70%). The two-replica limit capped further scaling.

Limitations. Both nodes share one laptop, so there is no cross-machine fault tolerance; `maxReplicas=2` and both replicas share the host CPUs, so scale-out adds little real capacity; metrics-server warms up slowly.

Task 5 — Subnetting & NACL

Subnets of `10.20.0.0/16` (largest first, contiguous):

Subnet	Hosts	CIDR	Range	Broadcast	Usable
app	1000	10.20.0.0/22	10.20.0.1–10.20.3.254	10.20.3.255	1022
public	250	10.20.4.0/24	10.20.4.1–10.20.4.254	10.20.4.255	254
data	100	10.20.5.0/25	10.20.5.1–10.20.5.126	10.20.5.127	126
management	20	10.20.5.128/27	10.20.5.129–10.20.5.158	10.20.5.159	30

NACL (public subnet). Verified **6/6** on the sample packets (`nacl_sim.py`). Inbound: deny all from `198.51.100.0/24` (rule 100, before the web allows), allow tcp 80/443 from anywhere, tcp 22 from management, tcp 1024–65535 from app (replies). Outbound: tcp 8000 to app, tcp 1024–65535 anywhere (replies). **Explanation.** A NACL is stateless, so replies need their own rules; and because the first matching (lowest-numbered) rule wins, the `198.51.100.0/24` deny must have a lower rule number than the 80/443 allows or packet S04 would be allowed.
Limitations / improvements. Largest-first contiguous allocation leaves gaps and a subnet can't grow in place; defining the plan as code (Terraform) would make it reproducible. A NACL is coarse (subnet-wide, stateless); per-instance stateful security groups would be finer.

Task 6 — Reproducibility, testing, CI/CD

`make up` builds and deploys from a fresh clone in two commands after the Release is available; `make down` deletes the cluster. CI stages: lint → tests (24 passed locally) → model evaluation from the `model-v1` Release → quality gate `score.py --min-validity 95` → gitleaks. The trained Release-model CI run passed all four jobs, including the 95% validity gate and a clean secret scan. A separate base-model run failed at Predict & score with 0% validity, demonstrating the gate. A separate fresh-clone k3d workflow passed build, deployment, Ingress, cold-start, load, rollout, recovery, and HPA steps. `AI_USE.md` records AI usage.

Limitations / improvements. The two k3d nodes share one laptop, and test coverage is not gated; a coverage threshold and a smoke deploy on every PR would strengthen validation.

Task 7 — Performance, cost, review

Load test (standard 2-minute Locust command): 4 users through the Traefik Ingress to two replicas (1 CPU / 1 GiB limit each): **254 requests**, **3 failures (1.18%)**, all HTTP 500 from the schema-checked model output. The p50 was **1500 ms**, p95 **3600 ms**, and throughput **2.155 rps** (`results/cluster_loadtest_stats.csv`). Pod peak samples were **623m CPU / 410 MiB** and **1003m CPU / 444 MiB**; cold-start median was 20 s. The p95 met the 5,000 ms target, but the 1% error target was *not* met.

Cost. *Assumption:* 2 replicas, each billed at limits 1 vCPU + 1 GiB. hourly = $2 \times (1 \times 3.50 + 1 \times 0.45)$ = **7.90 INR/hr**; at the measured **2.1550 rps** → 7,758.14 req/hr → cost/1000 = **1.0183 INR** (`results/cost_cluster.txt`).

SLO. *Target:* ≥ 99% of requests succeed over the load-test window. Cluster run: **251/254 (98.82%)** succeeded. The 1% error budget allowed 2.54 requests, i.e. at most 2 whole failures. The 3 failures exceeded it by 1 request.

Review table.

Component	Evidence (measured)	Limitation	Improvement
Data & storage	2748 kept of 3000	192 invented-slot replies	refine teacher prompt
Model quality	full_frame 50% (v1)	1 invalid intent/100	improve training targets
Latency & throughput	p95 3600 ms, 2.155 rps	CPU inference	request batching
Resource utilisation	1003m CPU, 444 MiB (one Pod)	shared host	independent nodes
Reliability & recovery	cold median 20 s, 3/254 load failures	model errors	stricter training objective
Security & cost	NACL 6/6, 1.0183 INR/1000	shared S3 identity	scoped keys

References

SLURP (Bastianelli et al., EMNLP 2020); Kim & Rush, Sequence-Level KD (EMNLP 2016); k3d, SeaweedFS, Hugging Face Transformers, Kubernetes, and AWS VPC NACL documentation.